

Algorithms and Data Structures

Problem Sheet 1

Hermann Schichl

1. Assume that your computer can execute 10^9 operations per second. For each of the following numbers of operations $f(n)$ and the given time limits calculate the (approximal) maximal size n of your problem that can be solved.

$f(n)$	1 μs	1 ms	1 s	1 min	1 $hour$	1 day	1 $week$	30 $days$	1 $year$	1000 $years$
$\lg \lg n$										
$\lg n$										
$\sqrt{n}$										
n										
$n \lg n$										
n^2										
n^3										
n^4										
2^n										
$n!$										

2. Consider the algorithm SUM_ARRAY from page 25 of the lecture book:

```

1: procedure SUM_ARRAY
2:    $sum = 0$ 
3:   for  $i = 1$  to  $n$  do
4:      $sum = sum + A[i]$ 
5:   end for
6: end procedure

```

It should solve the problem to sum the elements of the array A . Prove that the algorithm is correct by using a loop invariant.

3. We define the *searching problem* as follows: **Input:** A sorted sequence of n numbers $(a_1, a_2, \dots, a_n)$ with $a_i < a_{i+1}$ for alle $i < n$ provided in array A , a value x . **Output:** An index i such that $x = A[i]$ and if x does not appear in A , return the largest index i such that $A[i] < x$. If no such index exists, return 0. Write pseudocode for *linear search* that scans the array from the beginning to the end to determine i . Implement the procedure in Julia.

4. Using a loop invariant, prove that your algorithm from Exercise 3 is correct. Make sure that your loop invariant fulfills the three necessary properties. Give an asymptotic running time using the Landau symbols for the worst-case.
5. Let an array $A[1 : n]$ of numbers be given. Consider the following algorithm for solving the sorting problem: Find the smallest element of $A[1 : n]$ and then exchange it with the first element. Afterwards, find the smallest element in $A[2 : n]$ and exchange it with the second element, and so on. This algorithm is called *selection sort*. Write pseudocode for this algorithm and implement it in Julia.
6. Take the algorithm *selection sort* from Exercise 5 and find a loop invariant for that algorithm. Prove your assumption.
7. Analyse your algorithm from Exercise 5 and determine its best- and worst-case running times. Provide asymptotic running times using the Landau symbols (O , Ω , or Θ), whichever fit best.
8. Use mathematical induction to show that when $n = 2^k$ for $k \geq 1$, the solution of the recurrence

$$T(n) = \begin{cases} 2 & n = 2 \\ 2T(\frac{n}{2}) + n & n > 2 \end{cases}$$

is $T(n) = n \lg n$.

9. (a) Express $123n^2 + 345n - 148$ as Θ of a simple expression.
 (b) Is $2^{n+1} = O(2^n)$?
 (c) Is $2^{2n} = O(2^n)$?
 (d) Is $2^n = O(2^{2n})$?
 (e) Is $2^{n+1} = o(2^n)$?
 (f) Is $2^n = o(2^{2n})$?
10. Consider sequences $f(n)$, $g(n)$ and $h(n)$. Assume that $f(n) = \Theta(g(n))$ and that $g(n) = \Theta(h(n))$. Show that then $f(n) = \Theta(h(n))$.
11. For sequences $f(n)$, $g(n)$, $h(n)$, and $k(n)$ prove that $f(n) = \Omega(h(n))$ and $g(n) = \omega(k(n))$ implies $f(n) + g(n) = \Omega(h(n) + k(n))$
12. Let $f(n) = O(g(n))$ and $g(n) = o(h(n))$. Prove that $f(n) = o(h(n))$.
13. Use *Stirling's formula*

$$n! = \sqrt{2\pi n} \left(\frac{n}{e}\right)^n \left(1 + \Theta\left(\frac{1}{n}\right)\right).$$

to show that

- (a) $n! = o(n^n)$,
- (b) $n! = \omega(2^n)$,
- (c) $\lg(n!) = \Theta(n \lg n)$.

14. We define the *iterated logarithm function*

$$\lg^*(n) := \min\{i \in \mathbb{N} \mid \lg^{(i)}(n) \leq 1\},$$

where $\lg^{(1)}(x) := \lg x$, $\lg^{(k+1)}(x) := \lg(\lg^{(k)}(x))$. This function grows extremely slowly. Order the following functions by asymptotic growth:

$\lg(\lg^*(n))$	$2^{\lg^*(n)}$	$(\sqrt{2})^{\lg n}$	n^2	$n!$	$(\lceil \lg n \rceil)!$
$(\frac{3}{2})^n$	n^3	$\lg^2 n$	$\lg(n!)$	2^{2^n}	$n^{\frac{1}{\lg n}}$
$\lg \lg n$	$\lg^* n$	$n2^n$	$n^{\lg \lg n}$	$\log n$	1
$2^{\lg n}$	$(\lg n)^{\lg n}$	e^n	$4^{\lg n}$	$(n+1)!$	$\sqrt{\lg n}$
$\lg^*(\lg n)$	$2^{\sqrt{2 \lg n}}$	n	2^n	$n \lg n$	$2^{2^{n+1}}$

15. Use the substitution method to prove that the recursion

$$T(n) = 4T\left(\frac{n}{2}\right) + n$$

implies $T(n) = \Theta(n^2)$.

Hint: Subtract a lower order term.

16. For the recurrence

$$T(n) = 4T\left(\frac{n}{3}\right) + n$$

sketch the recursion tree and derive a guess for an asymptotic upper bound of its solution. Then use induction to prove your guess.

17. Consider the recursion describing Strassen's algorithm for matrix multiplication. Using the Master Theorem, prove that its running time $T(n)$ satisfies $T(n) = O(n^{\lg 7})$.